Linguagem de Programação
Ponteiros IV

Alexandre Mello

Fatec Campinas

2026

Roteiro

1. Exemplo de Ponteiros e TAD
2. Exercício

Implementando um TAD: Ponto

Iremos implementar um TAD que represente um ponto definido por suas coordenadas x e y.

Vamos introduzir o conceito de **tipo opaco**, que é ponteiro para a estrutura que define o ponto.

Além disso, também iremos definir um conjunto de funções que será visível para o programador que utilizará a biblioteca.

Implementando um TAD: Ponto

Tipo Opaco

Um tipo de dado é opaco quando ele é incompletamente definido em uma interface. Assim, os seus valores só podem ser acessados por funções específicas.

No arquivo **Ponto.h** iremos declarar tudo o que será visível para o programador.

Arquivo Ponto.h

```c
typedef struct ponto Ponto;

//Cria um novo ponto
Ponto* pto_cria(float x, float y);
//Libera um ponto
void pto_libera(Ponto* p);
//Acessa os valores "x" e "y" de um ponto
int pto_acessa(Ponto* p, float* x, float* y);
//Atribui os valores "x" e "y" a um ponto
int pto_atribui(Ponto* p, float x, float y);
//Calcula a distância entre dois pontos
float pto_distancia(Ponto* p1, Ponto* p2);
```

Implementação das funções

No arquivo **Ponto.c** iremos:

- Chamas as bibliotecas necessárias à implementação TAD
- Definir o tipo que define o acesso TAD
- Implementar as funções definidas no arquivo **Ponto.h**

Por estar definido dentro do arquivo **Ponto.c**, os campos dessa estrutura não são visíveis pelo usuário da biblioteca no arquivo **main.c**, apenas o seu nome definido no arquivo **Ponto.h**, que apenas declara um ponteiro para ele.

Arquivo main.c

```c
#include <stdio.h>
#include <stdlib.h>
#include "Ponto.h"
int main(){
    float d;
    Ponto *p,*q;
    //Ponto r; //ERRO
    p = pto_cria(10,21);
    q = pto_cria(7,25);
    //q->x = 2; //ERRO
    d = pto_distancia(p,q);
    printf("Distancia entre pontos: %f\n",d);

    pto_libera(q);
    pto_libera(p);

    return 0;
}
```

Arquivo Ponto.c

Definição do tipo de dados

```
#include <stdlib.h>
#include <math.h>
#include "Ponto.h" //inclui os Protótipos

struct ponto{
   float x;
   float y;
};
```

Arquivo Ponto.c

Alocação da TAD na memória e inicialização dos campos

```
Ponto* pto_cria(float x, float y){
    Ponto* p = (Ponto*) malloc(sizeof(Ponto));
    if(p != NULL){
        p->x = x;
        p->y = y;
    }
    return p;
}
```

Libera a memória alocada para um ponto

```
void pto_libera(Ponto* p){
    free(p);
}
```

Arquivo Ponto.c

Recupera, por referência, o valor de um ponto

```
int pto_acessa(Ponto* p, float* x, float* y){
    if(p == NULL)
        return 0;
    *x = p->x;
    *y = p->y;
    return 1;
}
```

Atribui a um ponto as coordenadas "x"e "y"

```
int pto_atribui(Ponto* p, float x, float y){
    if(p == NULL)
        return 0;
    p->x = x;
    p->y = y;
    return 1;
}
```

Arquivo Ponto.c

Calcula a distância entre dois pontos

```
float pto_distancia(Ponto* p1, Ponto* p2){
    if(p1 == NULL || p2 == NULL)
        return -1;
    float dx = p1->x - p2->x;
    float dy = p1->y - p2->y;
    return sqrt(dx * dx + dy * dy);
}
```

Exercício

Crie um TAD que represente um número complexo $z = x + iy$, em que $i^2 = -1$, sendo x a sua parte real e y a parte imaginária. O TAD deverá conter as seguintes funções:

- criar um número complexo;
- destruir um número complexo;
- soma de dois números complexos;
- subtração de dois números complexos;
- multiplicação de dois números complexos;
- divisão de dois números complexos.